

Sci-PL: A Mini Compiler for Scientific Computation

Eazdan Mostafa Rafin
Roll: 2003055
Section: A

November 13, 2025

1 Question 1: Lexical Analysis

This section presents the lexical analysis for the provided Sci-PL source code. The following table enumerates all lexemes identified in the source program, their corresponding token types, and the regular expression patterns that define them.

Pattern (Regular Expression)	Token Name	Lexeme
main	KW_MAIN	main
def	KW_DEF	def
if	KW_IF	if
else	KW_ELSE	else
while	KW_WHILE	while
check_balance	KW_FUNC_CHECK	check_balance
int	TYPE_INT	int
float	TYPE_FLOAT	float
bool	TYPE_BOOL	bool
reaction	TYPE_REACTION	reaction
[a-zA-Z_][a-zA-Z0-9_]*	IDENTIFIER	x, y, isBalanced, r1, r2
[0-9]+	LITERAL_INT	10, 1, 5, 2
[0-9]+\.[0-9]+	LITERAL_FLOAT	5.5, 1.0
"[^"]*"	LITERAL_STRING	"2H2+O2->2H2O", "H2+O2->H2O"
=	OP_ASSIGN	=
+	OP_PLUS	+
-	OP_MINUS	-
*	OP_MULTIPLY	*
>	OP_GT	>
<	OP_LT	<
&&	OP_LOGIC_AND	&&
{	DELIM_LBRACE	{
}	DELIM_RBRACE	}
(	DELIM_LPAREN	(
)	DELIM_RPAREN	)
;	DELIM_SEMICOLON	;
,	DELIM_COMMA	,

2 Question 2: Syntax Analysis

This section addresses the syntax analysis phase. We provide the Context-Free Grammar (CFG) and the Abstract Syntax Tree (AST).

2.1 Context-Free Grammar (CFG)

A simplified Context-Free Grammar (CFG) for the Sci-PL language is presented below.

$$Program \rightarrow KW_MAIN \ DELIM_LBRACE \ StmtList \ DELIM_RBRACE$$
$$StmtList \rightarrow Stmt \ StmtList \mid \varepsilon$$
$$\begin{aligned} Stmt &\rightarrow DeclStmt \\ &\mid AssignStmt \\ &\mid IfStmt \\ &\mid WhileStmt \end{aligned}$$
$$DeclStmt \rightarrow KW_DEF \ Type \ IdList \ DELIM_SEMICOLON$$
$$Type \rightarrow TYPE_INT \mid TYPE_FLOAT \mid TYPE_BOOL \mid TYPE_REACTION$$
$$IdList \rightarrow IDENTIFIER \ DELIM_COMMA \ IdList \mid IDENTIFIER$$
$$AssignStmt \rightarrow IDENTIFIER \ OP_ASSIGN \ Expr \ DELIM_SEMICOLON$$
$$\begin{aligned} IfStmt &\rightarrow KW_IF \ (Expr) \ \{ \ StmtList \} \ KW_ELSE \ \{ \ StmtList \} \\ &\mid KW_IF \ (Expr) \ \{ \ StmtList \} \end{aligned}$$
$$WhileStmt \rightarrow KW_WHILE \ (Expr) \ \{ \ StmtList \}$$
$$Expr \rightarrow LogicExpr$$
$$LogicExpr \rightarrow LogicExpr \ OP_LOGIC_AND \ RelExpr \mid RelExpr$$
$$\begin{aligned} RelExpr &\rightarrow ArithExpr \ OP_GT \ ArithExpr \\ &\mid ArithExpr \ OP_LT \ ArithExpr \\ &\mid ArithExpr \end{aligned}$$
$$ArithExpr \rightarrow ArithExpr \ OP_PLUS \ Term \mid ArithExpr \ OP_MINUS \ Term \mid Term$$
$$Term \rightarrow Term \ OP_MULTIPLY \ Factor \mid Factor$$
$$Factor \rightarrow Primary$$
$$Primary \rightarrow IDENTIFIER \mid LITERAL_INT \mid LITERAL_FLOAT$$
$$\mid LITERAL_STRING \mid CallExpr \mid DELIM_LPAREN \ Expr \ DELIM_RPAREN$$
$$CallExpr \rightarrow KW_FUNC_CHECK \ (Expr)$$

2.2 Parse Tree (CST)

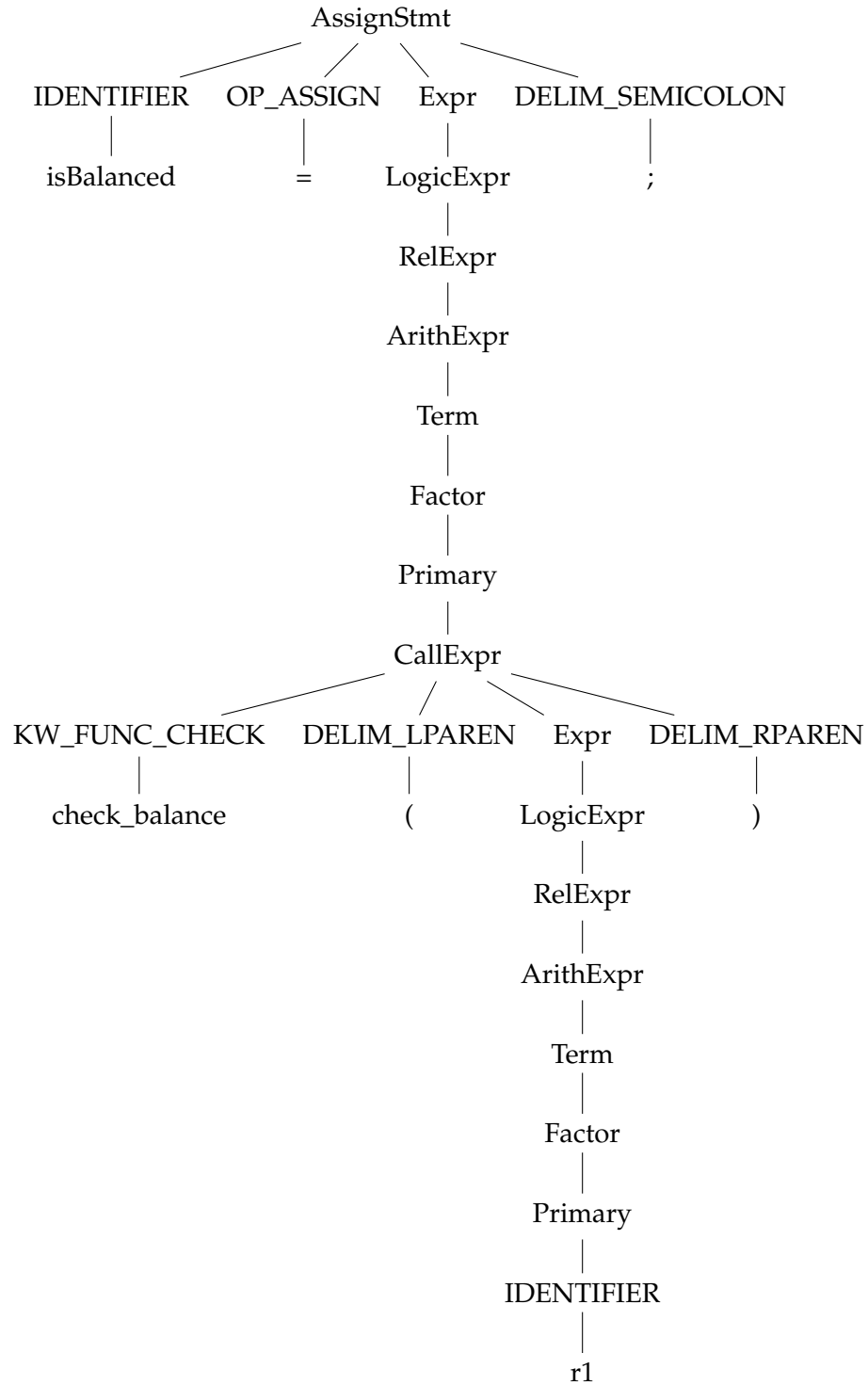


Figure 1: Parse Tree showing the full syntactic derivation.

2.3 Abstract Syntax Tree (AST)

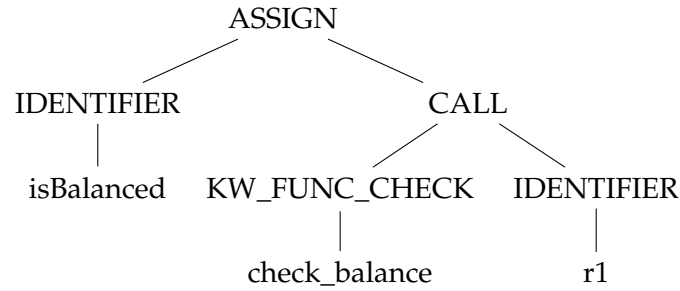


Figure 2: Abstract Syntax Tree (AST).

2.4 Explanation

The two figures illustrate different levels of structural representation used during the compilation process. Figure 1 shows the **Parse Tree**, which preserves the full syntactic detail of the input program, including every keyword, delimiter, operator, and grammar expansion. As a result, the Parse Tree is deep, verbose, and closely follows the grammar rules of the language.

In contrast, Figure 2 presents the **Abstract Syntax Tree (AST)**, a simplified and more compact representation that removes unnecessary syntactic elements such as parentheses, semicolons, and intermediate non-terminals. The AST retains only the essential semantic structure of the program. This makes it more suitable for subsequent compiler phases such as type checking, semantic analysis, and code generation.

3 Question 3: Intermediate Code Generation

This section provides the Three-Address Code (TAC) for the main operations of the Sci-PL source program. Temporary variables (e.g., `t1`, `t2`) are used to hold intermediate results from expressions, and labels (e.g., `L1`, `L2`) are used to manage the control flow for conditional and loop structures.

3.1 Three-Address Code (TAC)

This TAC adheres to several strict principles, many of which were illustrated in the target format:

1. **Atomicity:** Every complex statement is “atomized” into the smallest possible instructions. For example, a constant is first loaded into a temporary variable before being assigned.

- **Original:** `x = 10;`
- **TAC:**

```
t0 = 10
x = t0
```

2. **Temporaries for All Operations:** No variable is ever on the left-hand side of a complex expression. The result of any arithmetic or logical operation is always stored in a new temporary variable.

- **Original:** `y = y + 1.0;`
- **TAC:**

```
t11 = 1.0
t12 = y + t11
y = t12
```

3. **Explicit Control Flow:** All `if` and `while` statements are “unrolled” into conditional (`if_false`) and unconditional (`goto`) jumps, which explicitly manage the flow of execution.

- **If-Else (Lines 11-19):** An `if_false` statement is used to jump to the `else` block (labeled `L0`). The `if` block (the “true” case) ends with an unconditional `goto L1` to skip over the `else` block.
- **While Loop (Lines 20-27):** The loop is rebuilt with labels. `L2` marks the beginning of the loop (the condition check). `if_false t10 goto L3` checks the condition (`x > 5`) and jumps *out* of the loop to `L3` if it’s false. The `goto L2` at the end of the loop body unconditionally jumps back to the top to re-check the condition.
- **Logical AND (Lines 28-32):** The `&&` operator is implemented using **short-circuiting**. The code checks `isBalanced` first. If it’s false, it jumps to `L4` immediately, skipping the second check entirely. Only if the first check passes does it evaluate the second condition (`t16`).

The complete TAC generated from the source code is as follows:

```
1 t0 = 10
2 x = t0
3 t1 = 5.5
4 t2 = x + t1
```

```

5  y = t2
6  r1 = "2*H2 + O2 -> 2*H2O"
7  r2 = "H2 + O2 -> H2O"
8  t3 = CALL check_balance(r1)
9  isBalanced = t3
10 t4 = CALL check_balance(r2)
11 if_false t4 goto L0
12 t5 = 1
13 t6 = x + t5
14 x = t6
15 goto L1
16 L0:
17 t7 = 1
18 t8 = x - t7
19 x = t8
20 L1:
21 L2:
22 t9 = 5
23 t10 = x > t9
24 if_false t10 goto L3
25 t11 = 1.0
26 t12 = y + t11
27 y = t12
28 t13 = 1
29 t14 = x - t13
30 x = t14
31 goto L2
32 L3:
33 if_false isBalanced goto L4
34 t15 = 10
35 t16 = x < t15
36 if_false t16 goto L4
37 t17 = 2
38 t18 = y * t17
39 y = t18
40 L4:

```

4 Question 4: Assembly Code Generation

This code translates the Three-Address Code (TAC) into MASM assembly language.

```
1 ; --- 1. Setup and Includes ---
2 .686
3 .model flat, c
4 includelib C:\masm32\lib\msvcrt.lib
5
6 .stack 100 h
7 printf PROTO arg1 : Ptr Byte , printlist : VARARG
8 scanf PROTO arg2 : Ptr Byte , inputlist : VARARG
9
10 .data
11 r1 byte "2*H2_+_O2_->_2*H2O", 0
12 r2 byte "H2_+_O2_->_H2O", 0
13
14 ; locals: [ebp-4]=x, [ebp-8]=y, [ebp-12]=isBalanced
15
16 .code
17 main PROC
18     push ebp
19     mov  ebp, esp
20     sub  esp, 12                ; make locals
21
22 ; t0 = 10
23     mov  eax, 10
24
25 ; x = t0
26     mov  DWORD PTR [ebp-4], eax
27
28 ; t1 = 5.5    (use integer 5 for simplicity)
29     mov  eax, 5
30
31 ; t2 = x + t1
32     mov  edx, DWORD PTR [ebp-4]
33     add  eax, edx
34
35 ; y = t2
36     mov  DWORD PTR [ebp-8], eax
37
38 ; t3 = CALL check_balance(r1)
39     push OFFSET r1
40     call check_balance
41     add  esp, 4
42 ; isBalanced = t3
43     mov  DWORD PTR [ebp-12], eax
44
45 ; t4 = CALL check_balance(r2)
46     push OFFSET r2
47     call check_balance
48     add  esp, 4
49
50 ; if_false t4 goto L0
```

```

51     cmp eax, 0
52     je L0
53
54 ; t5 = 1
55 ; t6 = x + t5
56 ; x = t6
57     mov eax, DWORD PTR [ebp-4]
58     add eax, 1
59     mov DWORD PTR [ebp-4], eax
60     jmp L1
61
62 L0:
63 ; t7 = 1
64 ; t8 = x - t7
65 ; x = t8
66     mov eax, DWORD PTR [ebp-4]
67     sub eax, 1
68     mov DWORD PTR [ebp-4], eax
69
70 L1:
71 L2:
72 ; t9 = 5
73 ; t10 = x > t9
74     mov eax, DWORD PTR [ebp-4]
75     cmp eax, 5
76     jle L3
77
78 ; t11 = 1.0 (use 1)
79 ; y = y + t11
80     mov eax, DWORD PTR [ebp-8]
81     add eax, 1
82     mov DWORD PTR [ebp-8], eax
83
84 ; t13 = 1
85 ; x = x - t13
86     mov eax, DWORD PTR [ebp-4]
87     sub eax, 1
88     mov DWORD PTR [ebp-4], eax
89
90     jmp L2
91
92 L3:
93 ; if_false isBalanced goto L4
94     mov eax, DWORD PTR [ebp-12]
95     cmp eax, 0
96     je L4
97
98 ; t15 = 10
99 ; t16 = x < t15
100    mov eax, DWORD PTR [ebp-4]
101    cmp eax, 10
102    jge L4
103
104 ; t17 = 2

```



```
105 ; y = y * t17
106     mov eax, DWORD PTR [ebp-8]
107     shl eax, 1           ; multiply by 2
108     mov DWORD PTR [ebp-8], eax
109
110 L4:
111     mov esp, ebp
112     pop ebp
113     ret
114 main ENDP
115 end main
```